

Motion Smoothing and Alternate-Eye Rendering

Why a VR mod loses half its frame rate when the compositor starts smoothing, and how to fix it properly

A general engineering guide for VR mod authors

	Summary
Problem	Engine frame rate halves and head tracking stutters when the runtime enables Motion Smoothing / ASW
Cause	The mod blocks its game thread on the runtime's frame-pacing call, converting a submission-rate decision into an engine-wide throttle
Fix	Three changes: poll instead of block; capture on unsubmitted frames; locate poses every engine frame
Result	Full engine rate under an active lock, and double the previous per-eye rate
Scope	Any VR mod for a flat game, any runtime with a frame-pacing call

Contents

#	Chapter
1	What this document is
2	The symptom, precisely stated
3	Background: what motion smoothing actually does
4	Why AER mods are uniquely exposed
5	The OpenXR frame lifecycle, and where the trap is
6	The failure chain, step by step
7	First attempt, and why it failed
8	Second attempt, and the diagnostic that caught it
9	The fix, part one: never block the game thread
10	What the fix should produce
11	The fix, part two: everything downstream of the submit
12	A trap inside the fix: where to put the capture call
13	The fix, part three: head tracking
14	The complete solution
15	Instrumentation: what to measure
16	Applying this to a new mod: a checklist
17	What NOT to do
18	Why the per-eye rate doubles: the detailed argument
19	Interaction with the eye-pairing clock
20	Verifying the fix
21	Generalising beyond OpenXR
22	Closing: the meta-lesson
23	A worked timeline of one frame, before and after
24	Reference implementations worth reading
25	Frequently asked questions
26	Quick reference
27	Glossary
28	Appendix: the measurements from the real case

1. What this document is

This is a field guide to one specific, reproducible failure: a VR mod that runs at full frame rate until the runtime's motion smoothing is switched on, and then collapses to a quarter of the headset's refresh rate with head tracking that stutters in a regular sawtooth. It is written for someone building or maintaining a VR mod for a flat game, and it assumes no knowledge of the specific title that produced it.

The failure was diagnosed and fixed in a mod that converts a DirectX 11 Unreal Engine 3 game to stereoscopic OpenXR. The mod uses alternate-eye rendering (AER, also called AFR): the game has one rendering pipeline, so the mod renders one eye per engine frame and pairs consecutive frames into a stereo image. That architecture is extremely common in this class of mod, because almost no flat game exposes a second view.

Everything here generalises. The bug is not specific to Unreal Engine, to DirectX 11, or to the particular game. It is a consequence of how the OpenXR frame lifecycle interacts with a compositor that has decided to halve its frame rate, and any mod that waits on the runtime from its game thread will reproduce it.

Note. *The short version: never block the game thread on `xrWaitFrame`, and make sure nothing else in the mod is implicitly running at the submit rate. Those are two separate changes and the second is easy to miss.*

2. The symptom, precisely stated

The report that started the investigation was: 'when I turn on headset smooth motion the fps drops'. That is accurate but it understates the problem, and the understatement is part of why the first three attempts to fix it failed.

What was actually happening, measured:

Quantity	Smoothing off	Smoothing on
Engine frame rate	90/s	45/s
Frames submitted to the runtime	90/s	45/s
Fresh images per eye	45/s	22.5/s
Head tracking	smooth	regular stutter

Two things are worth noticing. First, the engine itself slowed down - not just the submission rate. A mod that merely submitted fewer frames would have kept its engine at 90. Second, head tracking degraded independently: the stutter had a regular period and was present even when looking around slowly, which is not what a frame rate drop alone looks like.

Those two observations turned out to be two different bugs with a shared root cause, plus a third bug that only became visible after the first was fixed. All three are covered here.

3. Background: what motion smoothing actually does

SteamVR calls it Motion Smoothing. Oculus calls it Asynchronous Spacewarp (ASW). Other runtimes have equivalents. The mechanism is the same in each case.

The compositor measures how long the application takes to produce a frame. If the application consistently misses the deadline for the headset's native refresh rate, the compositor concludes that the application cannot sustain it and takes over: it locks the application to half the refresh rate and synthesises the intermediate frames itself, using per-pixel motion vectors extracted from the last two application frames.

For a normal VR application this is a good trade. At a 90 Hz headset, the application now renders 45 real frames per second with a full 22 ms to do it in, and the compositor fabricates the other 45. The result is smoother than a stuttering 60-to-90 and it is what the feature exists for.

Note. *The critical property: once the compositor has decided to smooth, `xrWaitFrame` returns on the HALF-RATE cadence. It is not merely a hint. It is the runtime telling the application when it is allowed to produce a frame, and it will not return sooner.*

That last property is benign for a conventional application and catastrophic for an AER mod, for reasons the next chapter sets out.

4. Why AER mods are uniquely exposed

Under alternate-eye rendering, one engine frame carries one eye. The mod renders the left eye, then the right eye, then pairs them and submits a stereo image.

That means the rate at which each EYE receives a fresh image is half the engine's frame rate. At 90 engine frames per second, each eye is updated 45 times per second. This is the fundamental cost of AER and every AER mod pays it.

Now apply the compositor's half-rate lock on top:

normal VR app:	90 Hz	->	compositor locks to 45	->	45 Hz, both eyes
AER mod:	90 Hz	->	compositor locks to 45	->	22.5 Hz PER EYE

The halving compounds. A conventional application loses half its frame rate; an AER mod loses half of an already-halved per-eye rate. 22.5 Hz per eye is well below the threshold at which stereo fusion is comfortable, and it is why the symptom is so much more severe in this class of mod than the raw numbers suggest.

This much is inherent to AER and cannot be engineered away. What CAN be engineered away - and what this guide is about - is the second, entirely self-inflicted halving that occurs when the mod lets the compositor throttle the ENGINE as well as the submission.

5. The OpenXR frame lifecycle, and where the trap is

An OpenXR application's frame loop is a strict sequence:

```
xrWaitFrame(session, &waitInfo, &frameState);  
    -> blocks until the runtime says it is time to render  
    -> returns predictedDisplayTime and predictedDisplayPeriod  
  
xrBeginFrame(session, &beginInfo);  
    -> must be called exactly once per successful xrWaitFrame  
  
... render ...  
  
xrEndFrame(session, &endInfo);  
    -> submits the composition layers
```

The specification is explicit that `xrWaitFrame` is a throttling function. It exists so the runtime can pace the application. For a native VR application, being paced by the runtime is exactly right: the application has nothing else to do.

A mod is not a native VR application. A mod is a passenger inside a game that has its own frame loop, its own simulation tick, its own input handling and its own render thread. When the mod's hook blocks waiting for `xrWaitFrame`, it is not blocking a dedicated VR render thread - it is blocking THE GAME.

Note. *This is the whole bug in one sentence: `xrWaitFrame` is designed to throttle the caller, and in a mod the caller is the game engine.*

6. The failure chain, step by step

With that established, here is the exact sequence that produces the observed collapse.

Step	What happens
1	The compositor decides the app is missing deadlines and enables smoothing.
2	xrWaitFrame now returns on a ~22 ms cadence instead of ~11 ms.
3	The mod's Present hook calls xrWaitFrame and blocks for the full 22 ms.
4	The game's Present call therefore takes 22 ms to return.
5	The game's frame loop is paced by Present, so the ENGINE now runs at 45 fps.
6	AER halves that: each eye now gets a fresh image 22.5 times per second.
7	The game's own simulation, input and animation all slow to 45 Hz too.

Step 5 is the step that separates this bug from the inherent AER cost. The compositor never asked the engine to slow down. It asked the APPLICATION to submit frames at half rate. The mod converted that request into an engine-wide throttle by waiting for it on the game thread.

Step 7 is worth dwelling on because it is often the first thing a user notices without being able to name it: the game feels sluggish in a way that is not just visual. Animation and input sampling are running at half rate too.

7. First attempt, and why it failed

The first fix was to move `xrWaitFrame` onto a worker thread. The reasoning was sound: if `xrWaitFrame` blocks, do not call it from the game thread. A dedicated worker owns the wait; the game thread reads the result.

The implementation was a worker thread, an event signalled on completion, and a read of the cached frame state in the Present hook. This is a real and worthwhile optimisation - it removed a 13-18 ms per-frame stall on the game thread in the non-smoothed case.

It did not fix the smoothing collapse, because of one line:

```
WaitForSingleObject(g_evWaitDone, 200);
```

The game thread waited for the worker. The worker was waiting for `xrWaitFrame`. Under a half-rate lock, `xrWaitFrame` takes 22 ms to return, so the worker takes 22 ms, so the game thread waits 22 ms.

Note. *Moving a blocking call to another thread does not make it non-blocking if you then wait for that thread. It moves WHERE the block happens, not WHETHER it happens.*

This is an easy mistake to make and a hard one to see, because the architecture looks asynchronous. The worker is real, the event is real, the cached state is real. The only thing that is not asynchronous is the one call that matters.

8. Second attempt, and the diagnostic that caught it

The second fix added a fallback path: if the worker's result was not ready, extrapolate the display time locally and skip the submission rather than waiting.

The code was correct. It also never executed. The next test showed the frame rate still halved, and the instrumentation said why in three numbers on one line:

```
DIAG rate=45/s      the engine is pinned
runFree=0           the new fallback path NEVER fired
asyncStale=0        the async wait always SUCCEEDED
```

The fallback was guarded on the wait FAILING. The wait never failed - because the code waited for it. The guard was placed on the failure path when the block was on the success path.

Note. *A counter that stays at zero while the condition it guards against is visibly happening does not mean the guard is unnecessary. It means the guard is in the wrong place.*

This is the single most useful lesson in this document for debugging your own version of this bug. Instrument the fallback. If your anti-throttle path has a counter and that counter reads zero while the throttle is occurring, you have not fixed the throttle - you have written unreachable code.

9. The fix, part one: never block the game thread

The correct change is one line, and it is a change to the timeout:

```
// BEFORE - blocks the game thread for as long as the compositor wants
WaitForSingleObject(g_evWaitDone, 200);

// AFTER - poll; if the result is not ready, the engine runs on
const DWORD timeout = decoupled ? 0u : 200u;
WaitForSingleObject(g_evWaitDone, timeout);
```

With a zero timeout the game thread never waits. A frame whose wait has not completed simply does not get an XR frame; the engine continues and the next Present picks up the result.

The frame that ran without an XR frame still needs a display-time estimate for pose calculation. UEVR, praydog's universal Unreal Engine VR injector, states the pattern directly in its own source comments: the game thread can run multiple times before `xrWaitFrame` has finished, and the display time is extrapolated locally by one period when that happens.

```
if (waitNotReady) {
    frameState.predictedDisplayTime += frameState.predictedDisplayPeriod;
    ++framesRunFree;
    return;                // no XR frame this time; engine continues
}
```

Skipping the submission rather than submitting anyway is important for specification compliance: `xrBeginFrame` must be called exactly once per successful `xrWaitFrame`. A frame the runtime never granted is simply not begun. There is no violation and no state to reconcile.

10. What the fix should produce

After part one, under an active half-rate lock, the steady state should be:

Quantity	Value	Owner
Engine frame rate	full (90/s)	the game - never blocked
Frames submitted	half (45/s)	the compositor - its decision
Eye alternation	once per engine frame	the mod
Fresh images per eye	45/s	consequence of the above

The last row is the payoff and it deserves an explanation, because it is not obvious that the per-eye rate should double.

Two engine frames now fit between submissions. The eye alternates once per engine frame. So between one submission and the next, BOTH eyes have been rendered fresh. Each submitted stereo pair therefore carries two current images, rather than one current image and one a frame old.

Note. 45 Hz per eye under the same compositor lock that previously produced 22.5. The lock costs the submission rate, which is the compositor's to set. It no longer costs the engine rate on top, which never was.

11. The fix, part two: everything downstream of the submit

Part one alone produced a new symptom: the frame rate was stable and full, but one eye would freeze for a second or two, resume, and then the other eye would freeze.

Decoupling the engine from the compositor is only half a change. Everything in the mod that was implicitly running at the SUBMIT rate - because it happened to live downstream of the submission - is now running at the wrong rate relative to the engine.

In this mod the thing that mattered was the eye-slot capture. The code looked like this:

```
if (flatFrame) { ... }
else if (void* rtv = AcquireTargetRTV(&w, &h)) {
    ...
    ComposeEyes(rtv, w, h);    // <- performs the eye-slot capture
}
```

AcquireTargetRTV requires an XR frame to be open. On a run-free frame there is no XR frame, so it returns null, so ComposeEyes is never called, so the eye slot for that engine frame is never written.

Rate	Before part two
Engine	90/s
Eye alternation	90/s - alternates every engine frame
Submissions	45/s
Eye-slot captures	45/s - tied to the submission, not the engine

The eye alternates twice between captures, so the same eye tends to be the current one at each capture. The other eye is never written and freezes - until the cadence drifts by one frame and the roles swap. That is exactly the observed 'one eye freezes, then the other'.

The fix is to perform the capture regardless of whether an XR frame was begun. Capture needs only the swapchain, which the Present hook already has. It never needed the XR image; the coupling was incidental to where the call happened to sit in a branch chain.

12. A trap inside the fix: where to put the capture call

The first attempt at part two placed the capture as an else-branch alongside the existing one, which looked obviously correct and was dead code.

The reason is call ordering. In this mod the Present hook did roughly:

```
RunVrFrame(swapchain);      // contains the branch chain above
originalPresent(...);
XrVR_AfterPresent();        // contains the wait and the run-free decision
```

The run-free decision is made in AfterPresent, which runs AFTER RunVrFrame. So at the moment the branch chain executes, the XR frame is still open from the PREVIOUS AfterPresent. AcquireTargetRTV succeeds, and the else-branch is unreachable.

The instrumentation caught it immediately:

```
runFree=2749      the engine ran ahead 2749 times
capOnly=0         the run-free capture NEVER fired
```

Note. Place the run-free capture immediately after the code that makes the run-free decision, and trigger it by observing that decision's counter *CHANGE* - not by re-deriving the condition. A re-derived condition can disagree with the decision it is reacting to.

```
const auto freeBefore = FramesRunFree();
XrVR_AfterPresent();
if (FramesRunFree() != freeBefore)
    CaptureOnly();          // this frame ran free; fill the eye slot
```

13. The fix, part three: head tracking

With the frame rate stable and both eyes updating, one symptom remained: head tracking was not smooth. It stuttered with a regular period, and only with smoothing enabled.

The first theory was that the display-time extrapolation was accumulating. Adding one period per run-free frame means that over a long run of run-free frames the estimate drifts arbitrarily far ahead of any time the runtime will actually scan out. A bound was added.

The bound was correct and almost never engaged:

```
leadClamp=5      against      runFree=2990
```

Five clamps in three thousand frames is not a mechanism that could produce a visible, regular stutter. The extrapolation was not the problem, and the counter said so.

The actual cause was that the run-free path returned BEFORE the code that locates the head and eye poses:

```
line 4513:  return;                // run-free early return
line 4644:  xrLocateViews(...);    // never reached on those frames
```

On a run-free frame, the camera reused the pose from the last frame that received an XR frame. Under a half-rate lock that is every other frame: the head pose froze for one frame, then jumped two frames' worth when a real wait landed. A sawtooth - which is stair-stepped tracking on a perfectly stable frame rate.

Note. *xrLocateViews* is a *QUERY* against a *tracking space*. It is not part of the *frame lifecycle* and does not require an open frame. Call it every engine frame, unconditionally.

14. The complete solution

Three changes, in dependency order:

#	Change	Fixes
1	Poll the frame-wait with a zero timeout; never block the game thread	Engine throttled to half rate
2	Capture eye slots on run-free frames too, driven off the run-free counter	One eye freezing alternately
3	Locate views every engine frame, before any early return	Sawtooth head tracking

Each one is small. The difficulty is not in the code, it is in the diagnosis - and in particular in noticing that changes 2 and 3 are consequences of change 1 rather than unrelated bugs. Both were introduced, in effect, by fixing the first problem.

That is the general shape worth remembering: when you decouple two clocks that used to be the same clock, everything that silently depended on them being the same becomes a bug, and those bugs appear one at a time as each previous one is cleared.

15. Instrumentation: what to measure

Every one of the three bugs was found by a counter, and two of them were misdiagnosed at least once before the relevant counter existed. Add these before you start.

Counter	Meaning	Healthy value under a lock
runFree	frames where the engine ran without an XR frame	roughly half the present count
capOnly	eye captures performed on run-free frames	non-zero, similar order to runFree
cap0 / cap1	per-eye slot writes	within a few percent of each other
asyncStale	worker results rejected as stale	zero or near it
leadClamp	display-time extrapolations held back	small

The diagnostic value is in the RELATIONSHIPS, not the absolute numbers:

- runFree high but capOnly zero - the run-free capture is unreachable. This is bug two.
- cap0 and cap1 diverging - one eye is being starved. Also bug two.
- runFree zero while the engine is visibly throttled - the anti-throttle path is not firing at all. This is bug one, still present.
- leadClamp near zero while tracking stutters - the extrapolation is not your problem. Look at whether poses are being located at all on those frames. This is bug three.

Note. Print these on one line, on a timer, unconditionally. The single most valuable property of this instrumentation was that it fitted on one line and could be read at a glance from a user-supplied log.

16. Applying this to a new mod: a checklist

If you are building an AER-style VR mod for a flat game, work through this before you conclude that motion smoothing is simply incompatible.

- Find every call that can block the game thread on the runtime. In practice this is `xrWaitFrame`, any event or fence you wait on that is fed by it, and any acquire call on a swapchain image.
- Confirm the block by measurement, not by reading. Time the `Present` hook with smoothing on and off. If the hook's duration jumps to the smoothed frame period, you are being paced.
- Make the wait non-blocking. Poll with a zero timeout; on a miss, extrapolate the display time by one period and skip the submission for that frame.
- Verify with a counter that the run-free path actually fires. If it reads zero, stop and find out why before changing anything else.
- Audit everything downstream of the submission. Anything that runs only when an XR frame is open is now running at the submit rate while the engine runs faster.
- Locate poses every engine frame. Move `xrLocateViews` above every early return.
- Re-check per-eye capture parity. Two counters, one per eye; they should track.

17. What NOT to do

- Do not tell the user to disable motion smoothing and call it fixed. It is a legitimate feature and the mod's interaction with it is the mod's problem. This guide exists because 'nothing in the mod can override this, it is the compositor's decision' was written down as a conclusion and was wrong - it was true of the submit rate and got applied to the engine rate.
- Do not call `xrBeginFrame` on a frame you did not successfully wait for. The one-to-one pairing is a specification requirement, and violating it produces runtime-specific misbehaviour that is far harder to debug than the problem you started with.
- Do not attempt to defeat the compositor's decision - for example by forcing a submission rate. The compositor measured something real. The goal is to stop the decision propagating into the engine, not to overrule it.
- Do not assume moving a blocking call to a worker thread makes it asynchronous. Check whether anything then waits on that worker.
- Do not trust a fix you cannot see fire. Every one of the three fixes here was written correctly at least once and placed somewhere it could never execute.

18. Why the per-eye rate doubles: the detailed argument

This chapter justifies the claim in chapter 10, because it is the part most likely to be disbelieved.

Before the fix, with a half-rate lock, the engine is throttled to 45 fps. The eye alternates once per engine frame, so eye A is rendered at frame 1, eye B at frame 2, and so on. Each eye receives 22.5 fresh images per second. Every submission carries one image that was just rendered and one from the previous engine frame - which, at 45 fps, is 22 ms old.

After the fix, the engine runs at 90 fps and submissions occur at 45 per second. Between consecutive submissions there are two engine frames, and the eye alternates on each, so both eyes have been re-rendered. Each eye receives 45 fresh images per second, and the two halves of each submitted pair are 11 ms apart rather than 22 ms.

	Before	After
Engine rate	45	90
Submission rate	45	45
Fresh images per eye per second	22.5	45
Age gap within a submitted pair	22 ms	11 ms

The last row matters as much as the third. Under AER the two eyes of a pair are separated in TIME, and that separation is a false disparity that the brain interprets as depth. Halving the gap halves that error, so the fix improves stereo quality as well as frame rate.

19. Interaction with the eye-pairing clock

A related issue is worth flagging because it will bite anyone implementing AER, and it becomes much more visible once the engine and submission rates differ.

The eye must alternate on a signal that fires exactly once per engine frame. Deriving it from anything else produces intermittent wrong-eye images.

In the mod described here, the eye was originally advanced by the number of camera writes since the last present. On this particular game the camera is written twice per present, consistently. The code did:

```
frameEye = (frameEye + delta) & 1;          // delta == 2, always
```

and $(\text{eye} + 2) \& 1 == \text{eye}$. The eye never alternated at all. Both slots received the same viewpoint, forever - which presents as 'no stereo' rather than as a pairing bug.

Note. Use a signal the engine itself raises once per frame. A frame counter, a begin-frame hook, or a render-family entry point. Do not derive a once-per-frame decision from a signal whose rate you have not measured.

The commercially-developed VR mods examined during this work all do the same thing: one of them hooks the engine's 'begin engine frame' and 'begin render frame' as two separate events specifically so that eye assignment happens on a signal that fires exactly once per tick.

20. Verifying the fix

A fix of this kind is easy to believe in and hard to confirm. Use these tests.

- Toggle motion smoothing while the game runs and watch the engine frame rate. It should not change. The submission rate should halve.
- Check runFree. With smoothing on it should be approximately half the present count. With smoothing off it should be near zero.
- Check capOnly against runFree. They should be the same order of magnitude. If capOnly is zero while runFree is large, the run-free capture is unreachable.
- Check cap0 against cap1. They should stay within a few percent. Divergence means an eye is being starved.
- Look around slowly with smoothing on. Tracking should be as smooth as with it off. Any regular stutter means poses are not being located every frame.
- Confirm no runtime errors about frame ordering. If the runtime complains about `xrBeginFrame` without `xrWaitFrame`, the skip path is submitting when it should not.

21. Generalising beyond OpenXR

The specific API here is OpenXR, but the shape is not unique to it.

Any VR runtime provides a pacing call - OpenVR's `WaitGetPoses`, Oculus's `ovr_WaitToBeginFrame`, and the equivalent in vendor SDKs. All of them exist to throttle the caller, and all of them will happily throttle a game engine if a mod calls them from the game's thread.

The three-part structure of the fix transfers unchanged:

- Do not let the pacing call block the engine. Poll it, or run it on a worker you never wait on synchronously.
- Audit what silently depended on the pacing call's rate. Frame capture, resource acquisition and anything keyed to the submission are the usual suspects.
- Query poses on the engine's clock, not the compositor's. Pose queries are almost always free-standing and do not require an open frame.

The one API-specific caution is the frame-lifecycle pairing rule. OpenXR requires exactly one `xrBeginFrame` per successful `xrWaitFrame`. Check the equivalent constraint in your runtime before adopting the skip-the-submission approach.

22. Closing: the meta-lesson

The three bugs in this document took several iterations to fix, and in each case the code was written correctly before it was placed correctly. The failures were:

- A guard placed on the failure path when the block was on the success path.
- A capture placed before the decision it was supposed to react to.
- A pose query placed after an early return that skipped it.

None of those is a logic error. Every one of them is a placement error, and placement errors are invisible in review because the code reads correctly in isolation.

The only thing that caught them was instrumentation that reported whether the new code had actually executed. A counter on each fix, printed unconditionally, cost a few lines and resolved in one reading what inspection had failed to resolve across several attempts.

Note. *If you add a fix for a bug you cannot reproduce locally, add a counter that proves the fix ran. You will need it, and the person testing for you will be able to send you one line instead of a description.*

The underlying architectural point is simpler and worth stating on its own. A compositor is entitled to decide how often it will accept frames. It is not entitled to decide how often the game may render. Keeping those two rates separate is the entire fix, and everything else in this document is a consequence of having conflated them.

23. A worked timeline of one frame, before and after

It helps to see a single frame's worth of activity laid out. Assume a 90 Hz headset with smoothing active, so the compositor is granting frames every 22.2 ms.

Before the fix, the game thread is inside Present for the whole grant interval:

```
t=0.0 ms   game calls Present
t=0.0 ms   mod hook calls the frame wait
t=0.0 ms   ... blocked ...
t=22.2 ms  wait returns; mod captures an eye, submits
t=22.3 ms  Present returns to the game
t=22.3 ms  game runs its next simulation tick

ENGINE FRAME TIME: 22.3 ms  -> 45 fps
Eye A updated at t=0, eye B will update at t=44.6 -> 22.5 Hz per eye
```

After the fix, the wait is polled and the engine is never held:

```
t=0.0 ms   game calls Present; wait not ready; run free
           extrapolate display time; capture eye A; no submit
t=0.1 ms   Present returns; game ticks
t=11.1 ms  game calls Present; wait READY
           capture eye B; submit the pair
t=11.2 ms  Present returns; game ticks

ENGINE FRAME TIME: 11.1 ms  -> 90 fps
Both eyes updated within one 22.2 ms submission window -> 45 Hz per eye
```

Note what the compositor sees in both cases: one submission per 22.2 ms. Its decision is respected exactly. The difference is entirely on the engine side.

24. Reference implementations worth reading

Three existing mods were studied during this work. Each answered a specific question, and their approaches are worth knowing before designing your own.

Mod	Relevant technique
UEVR (praydog)	States the decoupling explicitly: the game thread runs multiple times before the frame wait completes, and the display time is extrapolated locally. Also the canonical plain-AER shake fix - the second eye reuses the first eye's rotation.
A commercial AnvilNext-based VR mod	Hooks 'begin engine frame' and 'begin render frame' as two SEPARATE events, so eye assignment happens on a signal that fires exactly once per tick and is ordered against submission.
A flat 3D/6DOF build of the same game	No compositor at all, so it is the control case: it shows which behaviours are inherent to the game and which are introduced by the XR layer. Its per-eye capture uses a boolean latch rather than a counter, which makes it immune to the twice-per-frame camera write described in chapter 19.

The general advice: when a symptom appears only in your mod and not in others of the same class, the difference is architectural rather than incidental, and finding the architectural difference is faster than debugging the symptom.

25. Frequently asked questions

- Does this fix let the mod run at full rate with smoothing on? The ENGINE runs at full rate. Submissions stay at the rate the compositor grants - that part is not ours to change and should not be.
- Is skipping a submission legal? Yes. The requirement is one `xrBeginFrame` per `SUCCESSFUL xrWaitFrame`. A frame that was never granted is simply not begun.
- Why not just call `xrWaitFrame` less often? The runtime's pacing is the point of the call; calling it less often does not make it return sooner and desynchronises the predicted display time from reality.
- Will this help without smoothing enabled? Somewhat - moving the wait off the game thread removes a per-frame stall that was measured at 13-18 ms on the test system. The dramatic improvement is specific to the smoothed case.
- Does it apply to non-AER mods? The engine-throttle half applies to any mod that waits on the runtime from the game thread. The per-eye halving is specific to AER.
- What if the run-free counter stays at zero? Then the game thread is still blocking somewhere. Look for other waits: swapchain image acquisition is the common second offender.

26. Quick reference

The whole fix, condensed.

```
// 1. never block the game thread
if (WaitForSingleObject(evt, decoupled ? 0 : 200) != WAIT_OBJECT_0) {
    frameState.predictedDisplayTime += frameState.predictedDisplayPeriod;
    ++framesRunFree;
    LocateViewsOnly();          // 3. poses every frame, before the return
    return;                    // no submit; engine continues
}

// 2. at the call site, capture on frames that ran free
const auto before = framesRunFree;
AfterPresent();
if (framesRunFree != before) CaptureEyeSlotOnly();
```

Symptom	Look at
Engine halves with smoothing on	the frame-wait timeout on the game thread
One eye freezes alternately	whether eye capture requires an open XR frame
Regular tracking stutter	whether pose location happens before an early return
No stereo at all	whether the eye actually alternates - measure the signal's rate
Fix appears to do nothing	add a counter and confirm the new code executes

27. Glossary

Term	Meaning
AER / AFR	Alternate-eye (or alternate-frame) rendering: one engine frame carries one eye, and consecutive frames are paired into a stereo image. Standard for mods of flat games, which have only one rendering pipeline.
Motion Smoothing / ASW	A compositor feature that locks the application to half the headset refresh rate and synthesises the intermediate frames from motion vectors.
Frame-pacing call	xrWaitFrame in OpenXR, WaitGetPoses in OpenVR. Blocks until the runtime is ready for the application to render. Designed to throttle the caller.
predictedDisplayTime	The instant the runtime expects the frame being produced to reach the display. Poses should be located for this time, not for now.
Run-free frame	An engine frame that proceeds without an XR frame, because the pacing call had not completed. The engine advances; nothing is submitted.
Eye slot	The mod's per-eye copy of a rendered frame, held until both eyes are ready to be submitted as a pair.
Reprojection	The compositor warping a submitted image from the pose it declares to the pose actually measured at scan-out.

28. Appendix: the measurements from the real case

For calibration, these are the actual numbers from the mod this guide is drawn from, on a 90 Hz headset with smoothing enabled, taken from a single session after all three fixes.

```
DIAG rate=90/s
runFree=5757      engine ran ahead 5757 times
capOnly=11284     run-free captures performed
cap0=10049 cap1=10101 per-eye slot writes, 0.5% apart
asyncStale=0      no stale worker results
leadClamp=8       extrapolation held back 8 times
```

Compare with the same counters before the second and third fixes:

```
DIAG rate=45/s      engine pinned
runFree=0           anti-throttle never fired    <- bug one present
...
runFree=2749        anti-throttle firing
capOnly=0           run-free capture unreachable  <- bug two present
...
runFree=2990
leadClamp=5         extrapolation not the problem <- bug three present
```

Each of those three lines was, in its session, the single piece of evidence that identified the next bug. None of them required a debugger, a capture tool or a reproduction on the developer's own machine - they were read from a log file sent by the person testing.

Note. That is the practical argument for this style of instrumentation. It converts remote debugging from a conversation about symptoms into a conversation about numbers.